

# COMPUTER PROGRAMMING

Electrical Engineering Department — UET Nowshera

---

## LAB 13

Lab Title: Structures in C++

### 1. OBJECTIVE

---

The purpose of this lab is to understand and implement structures in C++, including declaring structures that group variables of different data types, initialising structure variables, accessing members using dot notation, and working with multiple structure variables of the same type.

### 2. THEORY

---

A structure is a user-defined data type that collects variables of different types under a single name. Unlike an array in which all elements must be the same type, a structure can combine int, float, char, and other types as its members. The keyword struct is used to declare a structure.

Key characteristics of structures in C++:

- The closing brace of a structure declaration must be followed by a semicolon.
- A structure type declaration defines the form of the structure but does not allocate any memory. Memory is allocated only when a structure variable is declared.
- Individual members are accessed using the dot (.) operator on a structure variable.
- One structure variable can be assigned to another of the same type, performing a member-wise copy.
- One structure can be nested within another structure.
- Like an ordinary variable, a structure variable can be passed to a function either member by member or as a whole.

### 3. SOFTWARE USED

---

- DEV-C++ IDE
- C++ Compiler (bundled with DEV-C++)

## 4. PROCEDURE

---

### Task 1 — Structure Declaration and Initialization: Person

Declare a structure called Person with the following members: name (string), age (integer), and address (string). Declare a variable called person1 of type Person and initialize its members with values of your choice. Display the values of the members using dot notation.

```
#include <iostream>
using namespace std;

struct Person                                // declare structure
{
    char name[50];                            // member: name
    int age;                                  // member: age
    char address[100];                        // member: address
};

int main()
{
    Person person1 = {"Talha Ali", 19, "Dagai, Swabi"}; // initialize
    cout << "Name      : " << person1.name    << endl; // dot notation
    cout << "Age       : " << person1.age      << endl;
    cout << "Address  : " << person1.address  << endl;
    return 0;
}
```

#### Output:

```
Name      : Talha Ali
Age       : 19
Address  : Dagai, Swabi
```

### Task 2 — Structure Declaration and Initialization: Student

Declare a structure called Student with the following members: name (string), age (integer), and grade (char). Declare a variable of type Student called student1 and initialize its members with values of your choice. Display the values of the student1 structure members.

```
#include <iostream>
using namespace std;

struct Student                // declare structure
{
    char name[50];            // member: name
    int age;                  // member: age
    char grade;               // member: grade
};

int main()
{
    Student student1 = {"Talha Ali", 19, 'A'}; // initialize
    cout << "Name : " << student1.name << endl;
    cout << "Age : " << student1.age << endl;
    cout << "Grade : " << student1.grade << endl;
    return 0;
}
```

### Output:

```
Name : Talha Ali
Age : 19
Grade : A
```

### Task 3 — Accessing Structure Elements: Book

Declare a structure called Book with the following members: title (string), author (string), price (float), and pages (integer). Declare two variables of type Book called book1 and book2 and initialize their members with values of your choice. Display the structure members of both variables.

```
#include <iostream>
using namespace std;

struct Book                // declare structure
{
    char title[100];        // member: title
    char author[50];        // member: author
    float price;            // member: price
    int pages;              // member: pages
};

int main()
{
```

```

    Book book1 = {"C++ Programming", "Lafore", 850.0f, 500}; // first
book
    Book book2 = {"Data Structures", "Malik", 920.0f, 620}; // second
book

    cout << "--- Book 1 ---" << endl;
    cout << "Title : " << book1.title << endl;
    cout << "Author : " << book1.author << endl;
    cout << "Price : " << book1.price << endl;
    cout << "Pages : " << book1.pages << endl;

    cout << "--- Book 2 ---" << endl;
    cout << "Title : " << book2.title << endl;
    cout << "Author : " << book2.author << endl;
    cout << "Price : " << book2.price << endl;
    cout << "Pages : " << book2.pages << endl;
    return 0;
}

```

### Output:

```

--- Book 1 ---
Title : C++ Programming
Author : Lafore
Price : 850
Pages : 500
--- Book 2 ---
Title : Data Structures
Author : Malik
Price : 920
Pages : 620

```

## 5. CONCLUSION

---

This lab provided hands-on experience with structures in C++. Three structures — Person, Student, and Book — were declared, each containing members of different data types. Structure variables were initialised using brace-initialisation and their members were accessed and displayed using the dot (.) operator. The Book task demonstrated the use of two independent variables of the same structure type. The distinction between a structure type declaration (which defines form but allocates no memory) and a structure variable declaration (which allocates memory) was clearly understood.